

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

[Exercício: programação dinâmica](#)

Exercício: programação dinâmica

Encerrado

Fase de configuração	Fase de envio	Fase de avaliação	Fase de cálculo da nota da avaliação	Encerrado
	Envie seu trabalho Prazo limite dos envios: quinta, 24 nov 2022, 23:59 (1331 dias atrás)	Prazo limite da avaliação: quinta, 1 dez 2022, 23:59 (1324 dias atrás)		Fase atual ●

Conclusão

1. **(Impressão nítida)** Desejamos determinar em quais palavras realizar quebras de linha de modo a não sobrar muito espaço no final das linhas (exceto a última linha, que pode terminar com muitos espaços). Quando quebramos a linha em uma palavra, esta palavra vai iniciar a linha seguinte. Portanto, uma solução consiste em um subconjunto de palavras onde vamos realizar quebra de linha, e o custo de uma solução é a soma dos quadrados do número de espaços que sobram nos finais das linhas (exceto a última linha). Como entrada recebemos uma lista $L[1..n]$ contendo o número de caracteres de cada uma das n palavras no texto, e o número máximo W de caracteres que podem ser colocados em uma linha.

Para determinar o número de caracteres que sobram em uma linha que começa na palavra i e termina na palavra j , temos que calcular o número de caracteres $M[i, j]$ utilizados nesta linha. Podemos obter $M[i, j]$ somando os tamanhos das palavras de i até j , mais os $j - i$ espaços necessários para separar estas palavras. Ou seja, $M[i, j] = \sum_{k=i}^j L[k] + (j - i)$. Portanto, o custo desta linha vale $(W - M[i, j])^2$ se não for a última linha, ou vale zero se for a última linha.

Exemplo:

texto: "abcde fghijk lm nopqr stuv x w yz abcde fg hijkl"

índices: 1 2 3 4 5 6 7 8 9 10 11

tamanhos (L): 5 6 2 5 4 1 1 2 5 2 5

Tamanho máximo de uma linha: W = 12.

Qual o custo da solução que quebra as linhas nas palavras 3, 5, 8 e 11?

TEXTO	ÍNDICES	TAMANHO (M[i,j])	CUSTO (W-M[i,j])^2
abcde.fghijk	1 2	5+6+1=12	(12-12)^2 = 0
lm.nopqr....	3 4	2+5+1=8	(12-8)^2 = 16
stuv.x.w....	5 6 7	4+1+1+1+1=8	(12-8)^2 = 16
yz.abcde.fg.	8 9 10	2+5+2+2=11	(12-11)^2 = 1
hijkl.....	11	custo da última linha é sempre zero	

custo total: 0 + 16 + 16 + 1 = 33

Podemos preencher a matriz $M[i, j]$ em tempo $O(n^2)$ utilizando o algoritmo abaixo:

```
para i de 1 até n
  M[i,i] = L[i]
  para j de i+1 até n
    M[i,j] = M[i,j-1] + L[j] + 1
```

O algoritmo de backtrack recursivo abaixo fornece a solução ótima testando todas as soluções possíveis, mas possui complexidade de tempo exponencial. O algoritmo considera todas as possibilidades para a próxima quebra de linha, e resolve recursivamente a partir desta quebra de linha.

```
-----
Algoritmo impressão_nítida(M[1..n,1..n], W, i):
Entrada: M[a,b] é o tamanho da linha que começa na palavra a e termina na palavra b.
        W é o tamanho máximo de uma linha.
        i é o índice da primeira palavra.
Saída:  Tupla (sol,custo), onde 'sol' é o conjunto de índices de uma quebra linha ótima,
        e 'custo' é o custo desta solução ótima.

01  se M[i,n] <= W // última linha (caso base)
02      retorne ({}, 0)

03  menor_custo = oo
04  para j de i+1 até n // possíveis próxima quebra de linha
05      se M[i,j-1] > W, break
06      (sub_sol, sub_custo) = impressão_nítida(M,W,j)
07      custo = (W - M[i,j-1])^2 + sub_custo
08      se custo < menor_custo
09          menor_custo = custo
10          melhor_solução = sub_sol U {j}
11  retorne (melhor_solução, menor_custo)
-----
```

Com base no algoritmo de backtrack recursivo acima, projete um algoritmo de programação dinâmica de tempo polinomial para o mesmo problema.

- Qual o número de subsoluções armazenadas na tabela de programação dinâmica? Justifique.
- Como é feito o preenchimento do caso base?
- Qual a ordem de preenchimento da tabela? Argumente que esta ordem não acessa elementos não preenchidos.
- Forneça o psedocódigo.
- Qual a complexidade de tempo?

Obs.: Como a linha 10 do backtrack recursivo realiza uma cópia da lista `sub_sol`, a complexidade depende do número de elementos em `sub_sol`. Entretanto, é possível recuperar a solução ótima armazenando apenas a melhor próxima quebra de linha `j` para cada posição `i` de início do texto. Desta forma, para simplificar podemos considerar que cada execução da linha 10 tem tempo constante.

Seu envio ▼

Você não enviou seu trabalho ainda

◀ [Vídeo: programação dinâmica \(58min\)](#)

Seguir para...

[Slides: programação dinâmica ▶](#)

©2020 – Universidade Federal do Ceará – Campus Quixadá.

Todos os direitos reservados.

Av. José de Freitas Queiroz, 5003

Cedro – Quixadá – Ceará CEP: 63902-580

Secretaria do Campus: (88) 3411-9422

 [Baixar o aplicativo móvel.](#)